

Payrail

Structure, architecture and logic — snapshot as of 30 September 2026 (Day 5)

Payrail is an early-stage FastAPI service of roughly 60 lines of real code. Judging by its names (quotes, providers, Money, send and receive amounts), it is a **cross-border payment quote aggregator**. A client asks “I want to send X of currency A from country B to country C”, Payrail asks several payment providers what that would cost and returns their quotes.

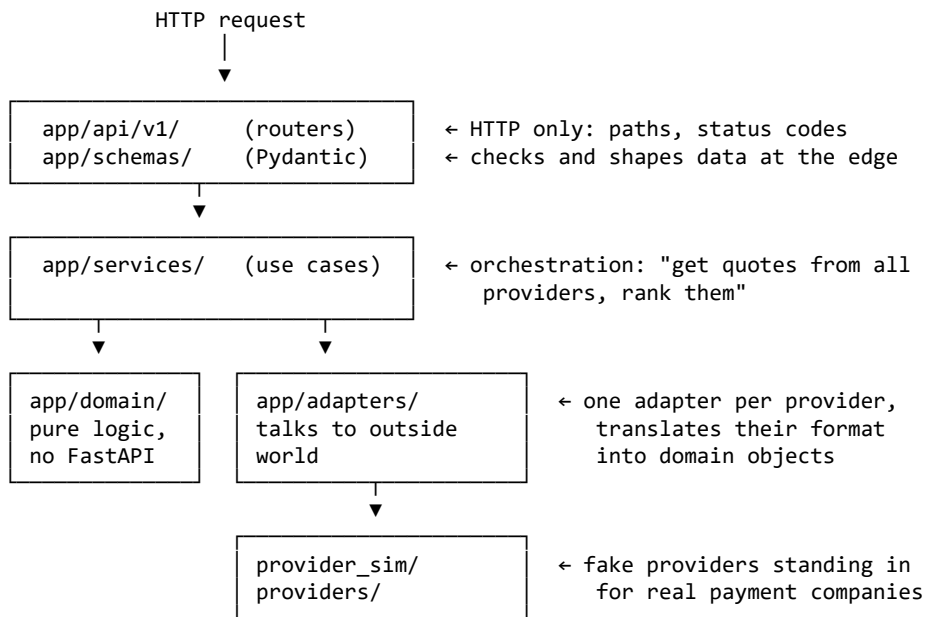
This document separates what is **built** from what the folder layout **plans**, because most folders are still empty placeholders.

1. Tooling

File	Role
pyproject.toml	Project metadata. The only runtime dependency is fastapi[standard], which brings in Pydantic, Uvicorn and the fastapi CLI. The dev group has pytest, pytest-asyncio, httpx (used by TestClient) and ruff.
uv.lock / .python-version	Managed by uv , pinned to Python 3.13. uv sync installs the exact locked versions.
.gitignore	Standard Python ignores.
.claude/CLAUDE.md	Working rules for Claude: explain, review and question; no implementation code in domain, services, adapters, schemas or test assertions.
docs/log.md	Daily learning journal.

2. Architecture: layers and dependency direction

The folders follow a **layered (hexagonal / “ports and adapters”)** architecture. The core rule is that dependencies point inward:



app/db/ → persistence (quotes, audit log) – later
 app/middleware/ → cross-cutting: request IDs, logging, timing – later
 tests/unit/ → domain + services, no HTTP
 tests/integration/ → full HTTP round-trips

The main split is between **schemas** and **domain**:

- `schemas/` is the *contract with the outside world*. Pydantic models validate untrusted JSON: types, lengths, ranges. They belong to the API layer.
- `domain/` is *your business's own vocabulary*. Plain frozen dataclasses that know nothing about HTTP, JSON or FastAPI, reusable unchanged in a CLI or background worker.

The Day 5 note “Confused `schemas/quote.py` with `api/v1/quotes.py`” is really about this boundary: `schemas/quote.py` describes *what a request looks like*; `api/v1/quotes.py` says *which URL accepts it and what happens next*.

3. What's built, file by file

app/main.py — the composition root

- Creates the single `FastAPI()` app object that Uvicorn serves.
- `include_router(router, prefix="/v1")` mounts every route from the quotes router under `/v1`. The router declares `/quotes`, so the final path is `POST /v1/quotes`. This is API versioning: a future `/v2` would be a separate router and wouldn't break existing clients.
- `GET /health` is a liveness check. Load balancers, Docker and Kubernetes call it to ask “is the process up?”

The Day 5 bug fits here: writing `router = APIRouter()` again in `main.py` replaced the imported router (which had the route on it) with an empty one. A Python name points to whatever was assigned to it last.

app/api/v1/quotes.py — the HTTP layer

`APIRouter()` is a mini-app: a group of routes that `main.py` plugs in. Because `create_quote(payload: QuoteRequest)` type-hints a Pydantic model, FastAPI will:

- 1 read the JSON body,
- 2 build a QuoteRequest from it,
- 3 if that fails, **automatically return 422** listing every field error, without the function ever running,
- 4 otherwise call the function with a validated object.

Right now it just **echoes the request back**, which is why a valid call returns 200 with the same body.

app/schemas/quote.py — the input contract

- `amount_minor: int, ge=1`. Amount in **minor units** (cents, paisa), so 10.50 USD is 1050. Standard in payments because floats can't represent decimal money exactly ($0.1 + 0.2 \neq 0.3$). `ge` = greater than or equal to, so the amount must be positive.
- `source/target_currency` — exactly 3 characters, the shape of ISO 4217 codes (USD, PKR).
- `source/target_country` — exactly 2 characters, the shape of ISO 3166-1 alpha-2 codes (US, PK).

app/domain/models.py — the business vocabulary

- `Money(amount_minor, currency)` is a *value object*. An amount without its currency is meaningless, so the two always travel together — preventing bugs like adding USD cents to PKR paisa.
- `Quote` is one provider's offer: who (`provider`), what you pay (`send_amount`), what arrives (`receive_amount`), what they charge (`fee`) and how long it takes (`delivery_hours`).
- `frozen=True` makes instances immutable: once a provider has quoted, nothing downstream can quietly change the numbers. Frozen dataclasses are also hashable (usable in sets or as dict keys).
- Not used anywhere yet — waiting for the services and adapters layers.

tests/test_health.py

`TestClient` wraps the app in memory, so no server needs to run. It sends a real HTTP-shaped request and checks the status code and JSON. Run with `uv run pytest`.

4. The intended request flow (not built yet)

The log's next step is "the provider simulator — three fake payment providers". That points to this flow:

- 1 `POST /v1/quotes` receives JSON; Pydantic validates it into a `QuoteRequest`.
- 2 The router hands the request to a **service** in `app/services/`.
- 3 The service calls each **adapter**, one per provider, probably concurrently — where `async` starts to matter.
- 4 Each adapter calls its provider in `provider_sim/`, gets that provider's own format and turns it into a domain `Quote`.
- 5 The service collects the `Quotes`, may drop failures or timeouts, and ranks them by what the receiver actually gets.
- 6 The router converts domain `Quotes` into a **response schema** (a second Pydantic model) and returns JSON.

Later, `db/` could save quotes and `middleware/` could add request IDs and logging.

5. Questions worth thinking about

- 1 **Validation depth.** Would "usd", "123" or "U\$D" pass as a currency right now? Should they? Look at what else `Field` accepts besides lengths.
- 2 **Same-currency requests.** What should happen when `source_currency == target_currency`? A single field constraint can't express a rule involving two fields — look up Pydantic's *model validators*.
- 3 **Response shape.** What should the endpoint return once quotes exist? Where would that model live, and why not return domain dataclasses directly?
- 4 **Money invariants.** `Money(-500, "xx")` constructs without complaint. Should the domain refuse invalid money on its own, without relying on the schema layer?
- 5 **Test coverage.** No test yet for `/v1/quotes` (neither 200 nor 422). `tests/integration/` is the natural home.
- 6 **Housekeeping.** `README.md` is empty, `pyproject.toml` has the placeholder description, and `docs/log.md` has two "Day 5" entries.

In short

Payrail is a versioned FastAPI app with layers kept separate. Pydantic schemas guard the edge, and frozen dataclasses model the business core. Services, adapters and a provider simulator will connect the two. Right now the edge works (validation and routing) and the core types exist, but nothing connects them yet — that connection is the provider simulator planned next.